

Serverless Architecture

Serverless Architecture

- Neste estilo de arquitetura a criação e execução das aplicações ou módulos que compõem o sistema são abstraídos, não sendo necessário o gerenciamento dos servidores.
- Em contrapartida, é preciso configurar serviços na nuvem que serão gerenciados pela própria nuvem, por exemplo, o provisionamento, escalonamento e manutenção dos servidores é responsabilidade da nuvem hospedeira, não do time de desenvolvimento da aplicação.
- Como pode ser percebido, esse tipo de arquitetura é altamente escalável.

Serverless Architecture

- Backend como Serviço (BaaS)
 - Serviços prontos para funcionalidades específicas, como autenticação, banco de dados, notificações e armazenamento de arquivos.
 - Firebase, Supabase ou AWS Amplify.
- Funções como Serviço (FaaS)
 - Funções isoladas que são executadas sob demanda em resposta a eventos (como requisições HTTP ou alterações em um banco de dados).
 - AWS Lambda, Google Cloud Functions, ou Azure Functions.

Serverless Architecture

- Custo-Eficiência
 - Você paga apenas pelo tempo de execução do código e pelos recursos usados, sem custos fixos para manter servidores ociosos.
- Escalabilidade Automática
 - A infraestrutura ajusta automaticamente a quantidade de recursos para atender à demanda, desde picos altos até períodos de baixa atividade.
- Redução de Sobrecarga Operacional
 - Não é necessário configurar, monitorar ou atualizar servidores manualmente.
- Desenvolvimento mais Rápido
 - Você pode se concentrar na lógica do negócio em vez de se preocupar com infraestrutura.

Serverless Architecture

- Pontos não tão bons...
- Cold Starts
 - As funções serverless podem levar mais tempo para responder na primeira execução após ficarem inativas por um período. Precisam ser “levantadas” dado que estavam adormecidas para economizar recursos.
- Dependência de Provedor
 - Pode ser difícil migrar sua aplicação para outro provedor, pois muitos serviços utilizam APIs específicas. Por exemplo, AWS Lambda só vai funcionar na AWS :-). Embora você possa migrar o código não é algo tão fácil como subir um container Docker que independe do servidor.
- Monitoramento e Debugging
 - A natureza distribuída da arquitetura pode tornar o rastreamento de problemas mais complexo. Imagine ter que ler vários logs de diferentes serviços para saber onde quebrou e qual o caminho que foi feito pela requisição.

Serverless Architecture

- Exemplo sistema Serverless
- Visualizar Produtos
 - O frontend React faz uma requisição para uma função serverless (ex.: Lambda), que consulta o banco de dados e retorna a lista de produtos.
- Adicionar um Produto
 - O usuário insere os detalhes no frontend.
 - Uma requisição POST é feita para uma função que salva os dados no banco de dados.
- Editar Produto
 - Atualizações são feitas via uma requisição PUT para outra função serverless.
- Excluir Produto
 - Uma requisição DELETE aciona uma função para remover o produto do banco e sua imagem do armazenamento.
-

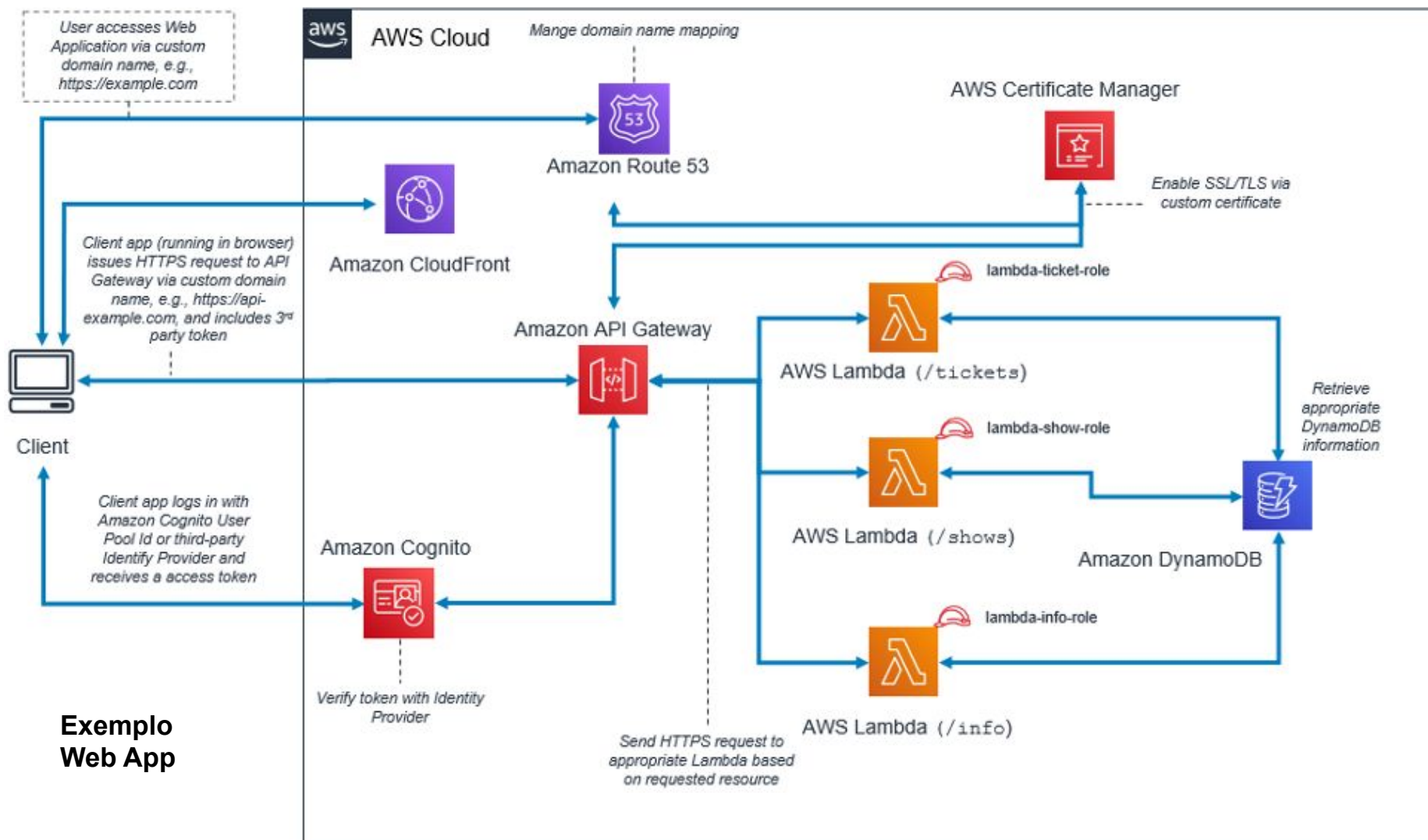
Serverless Architecture

- **Ferramentas e Tecnologias Sugeridas**
- Hospedagem do Frontend
 - Vercel, Netlify ou AWS Amplify.
- Backend
 - AWS Lambda, Google Cloud Functions, ou Firebase Cloud Functions.
- Banco de Dados
 - Firebase Firestore, DynamoDB, Aurora Serverless ou Atlas DB (mongo).
- Armazenamento de Imagens
 - AWS S3, Google Cloud ou Firebase Storage.
- Autenticação
 - Firebase Authentication ou AWS Cognito.

Serverless vs Microservices Architectures

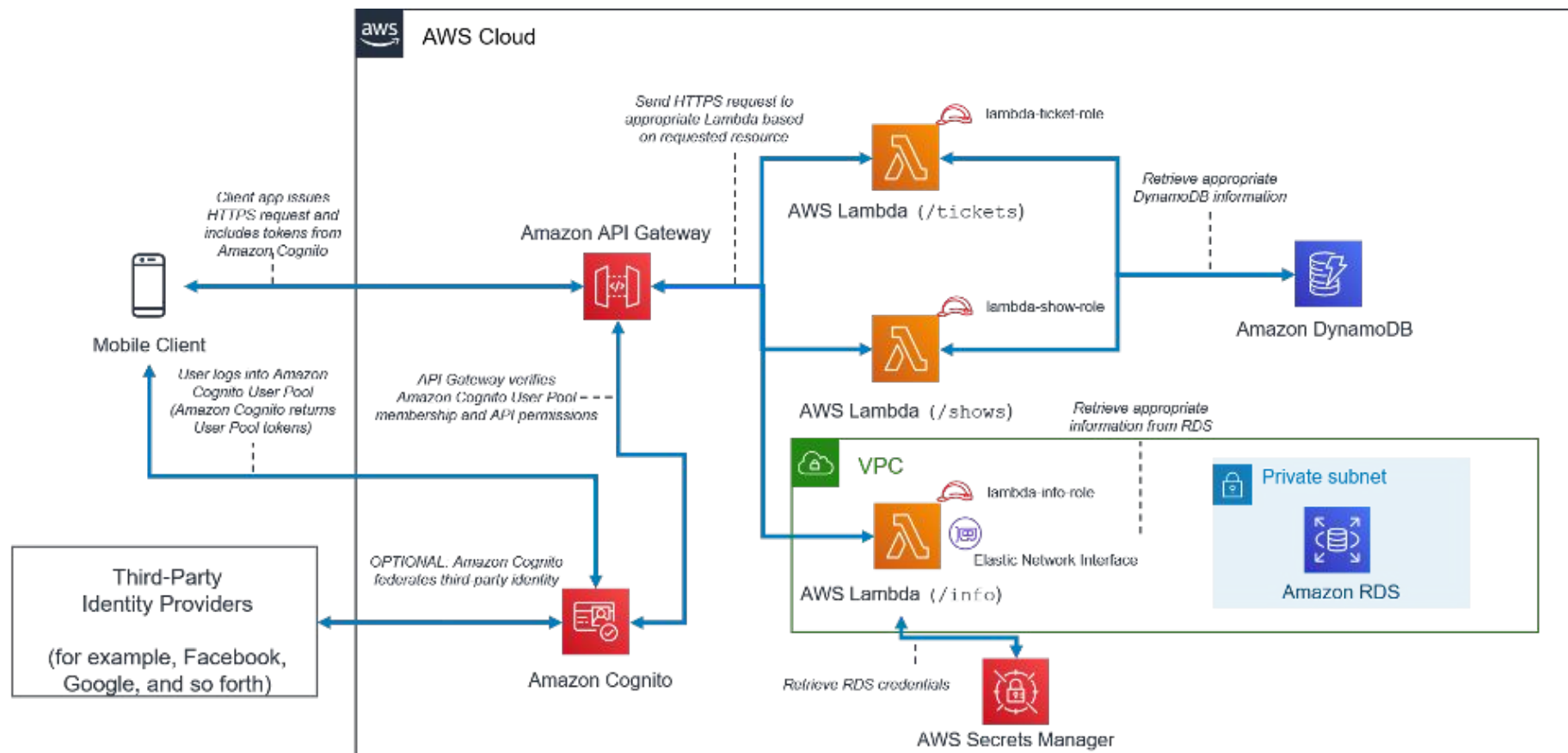
Aspecto	Microserviços	Serverless	Relação
Unidade lógica	Serviço	Função	Uma função pode ser vista como o "menor tipo" de microserviço
Infraestrutura	Containers (gerenciados por você)	Funções (gerenciadas pela nuvem)	Serverless abstrai infraestrutura
Escalabilidade	Manual ou via orquestrador (ex: Kubernetes)	Automática por evento	Serverless é mais granular
Estado	Pode ser stateful (banco acoplado)	Stateless (dados externos)	Funções precisam reconstituir contexto a cada execução
Comunicação	REST, gRPC, mensageria	Event-driven (filas, tópicos, gatilhos)	Serverless é fortemente orientado a eventos
Custos	Instâncias 24/7	Pay-per-use	Serverless é mais econômico para baixa demanda
Observabilidade	Log centralizado (ELK, Prometheus)	Logs distribuídos (CloudWatch, Stackdriver)	Difícil rastrear em Serverless
Gerenciamento	DevOps / SRE	Cloud provider	Serverless delega operação à nuvem

Serverless Architecture



<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

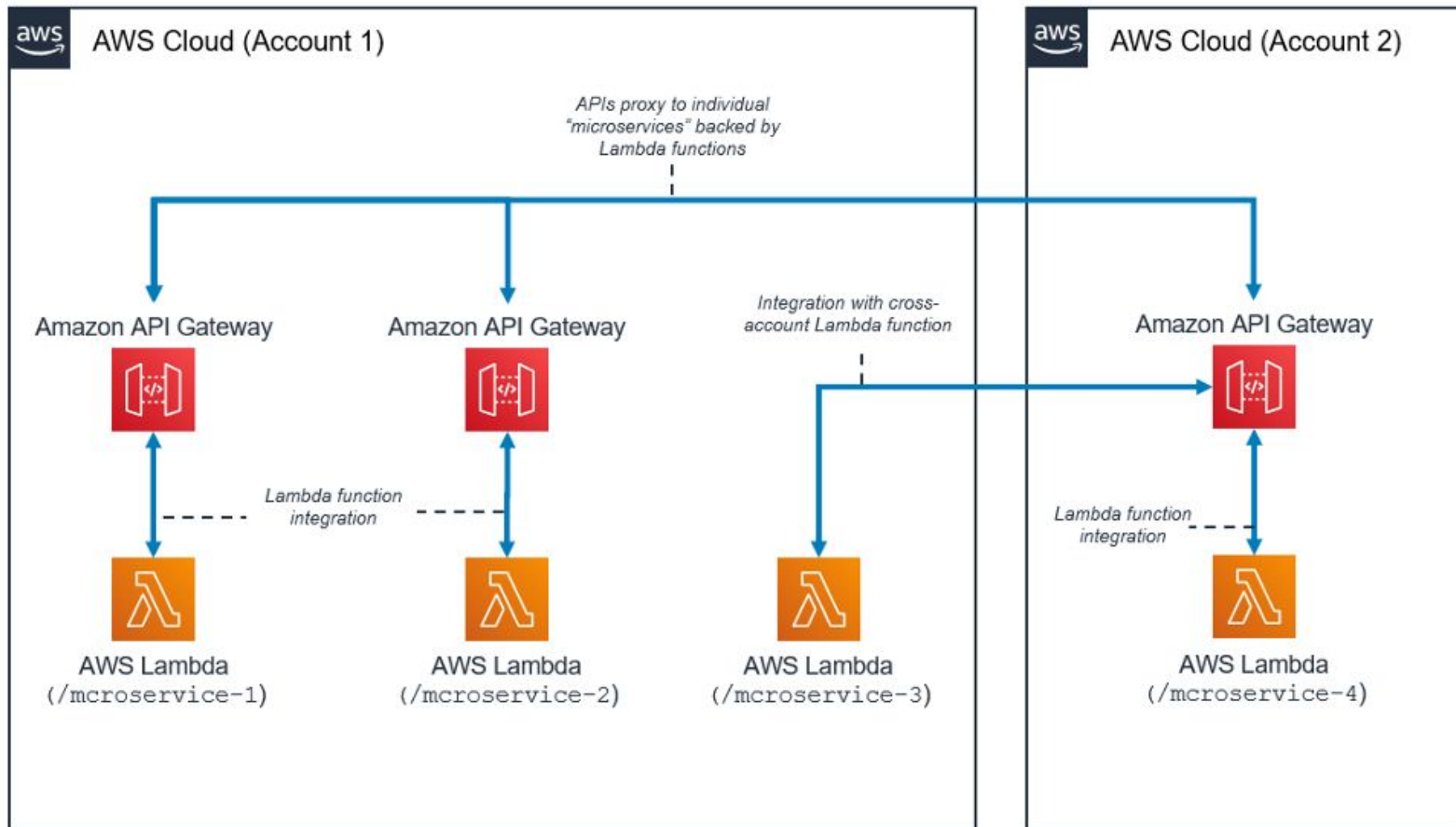
Serverless Architecture



**Mobile
backend**

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

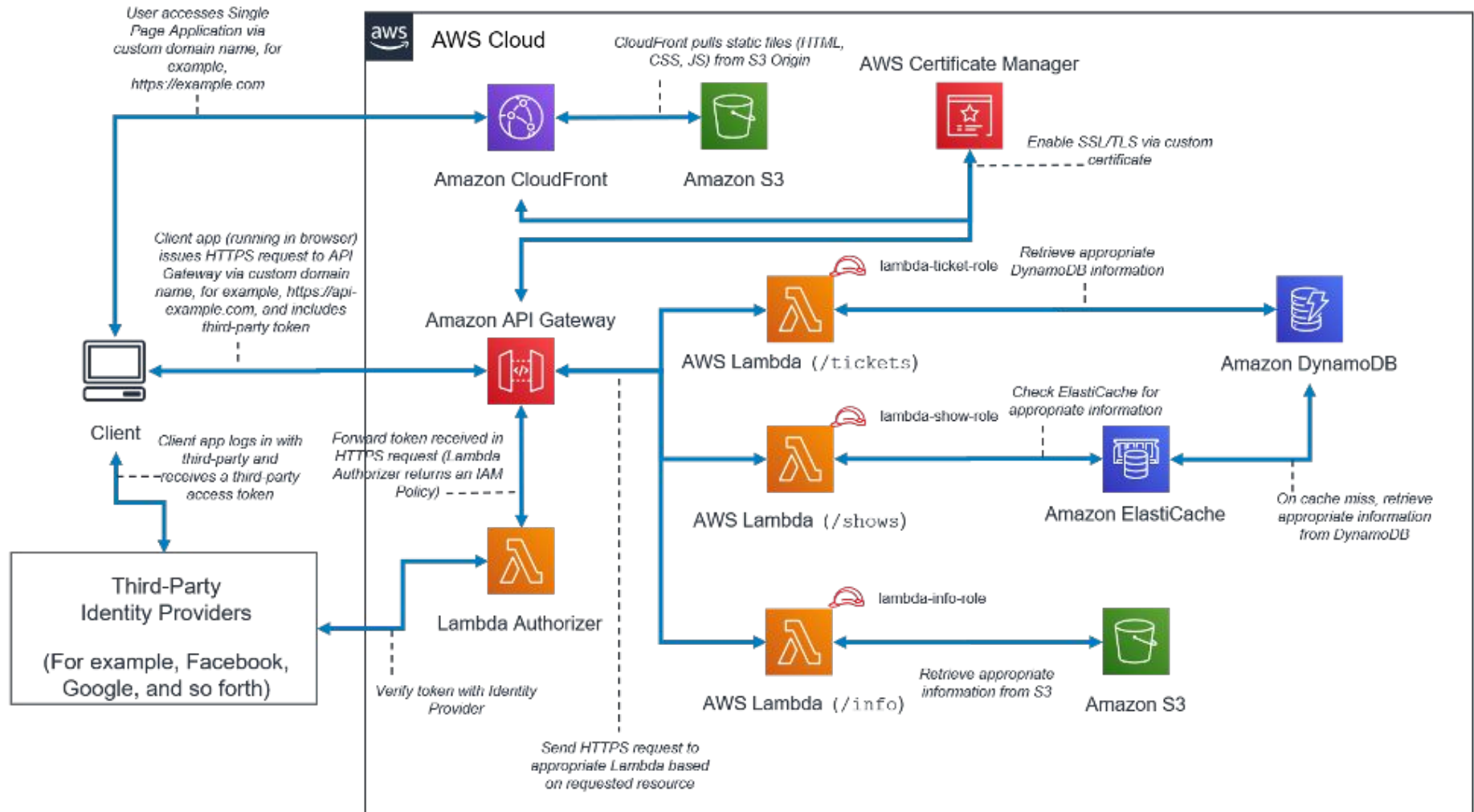
Serverless Architecture



Microserviços com lambda

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

Serverless Architecture



SPA

<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/web-application.html>

Serverless Architecture

- **E o que é o API Gateway da AWS?**
- O AWS API Gateway é um serviço gerenciado da Amazon Web Services que permite aos desenvolvedores criar, publicar, manter, monitorar e proteger APIs em escala.
- Ele é amplamente usado para expor endpoints de APIs REST, WebSocket e HTTP, possibilitando que diferentes aplicativos (web, móveis, etc.) se comuniquem de forma segura e eficiente com os serviços de backend.
- No contexto das arquiteturas serverless, pode-se criar um API Gateway para receber as requisições do usuário e mapear a requisição para uma determinada função lambda, chamando a função lambda de forma encapsulada, sem o conhecimento do usuário.

Serverless Architecture

- Exemplo prático
 - Máquinas de Venda Coca-cola
 - O cliente pode fazer o pedido no aplicativo do celular ou na própria máquina de refrigerantes.
 - Ao realizar o pedido, é iniciado um processo de autorização do crédito e liberação do refrigerante pela máquina.
 - Parece uma aplicação bem simples, certo?
 - E é, mas exige uma grande infraestrutura pela quantidade de requisições.
 - Quantas pessoas no mundo (ou em um país específico) estão comprando refrigerantes ao ir ou voltar do trabalho? Ao estar em passeio de lazer?



Serverless Architecture

- Ao migrar para uma arquitetura serverless na AWS, foi alcançado uma redução de 40% nos custos com infraestrutura.

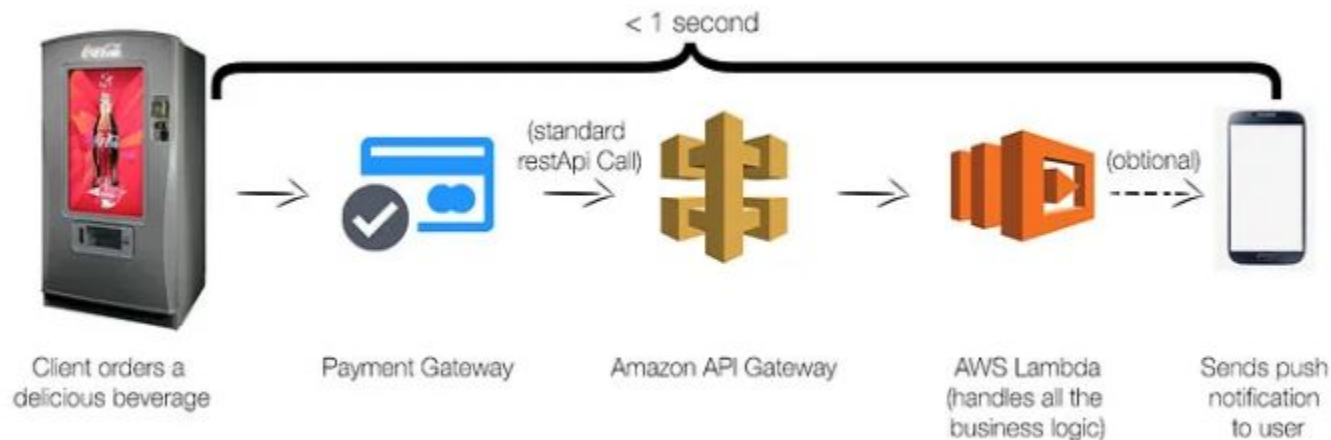
 Serverless in AWS				
	Managed NAT	replaces	Amazon EC2	costs \$0.045 per hour
	Lambda	replaces	EC2	costs \$0.00001667 per GB-s
	API Gateway	replaces	apigee, mashery	costs \$3.50 per MM calls + data
	Amazon SQS (queue)	replaces	messaging	costs \$0.60 per MM messages
	Amazon SES (email)	replaces	email providers	costs \$0.10 per 1000 messages
	Amazon SNS (notif.)	replaces	mobile provider	costs \$0.50 per MM SMS messages
	Amazon DynamoDB	replaces	big databases	costs \$180/mo for 36K writes per sec
	Amazon RDS (DB)	replaces	EC2 DBs	costs vary based on capacity

Serverless Architecture

- As máquinas automáticas de vendas de refrigerantes espalhadas pelo mundo possuem uma conexão com a central de processamentos da Coca-Cola.
- A lógica é bem simples, o cliente faz o pedido de uma bebida, o pagamento é processado pela Coca-Cola ou um parceiro que inicia uma requisição HTTP para uma API para liberar a bebida.

Serverless Architecture

- A Lambda da AWS é responsável por toda a lógica de negócio, similar ao back-end necessário para um processamento não-serverless.

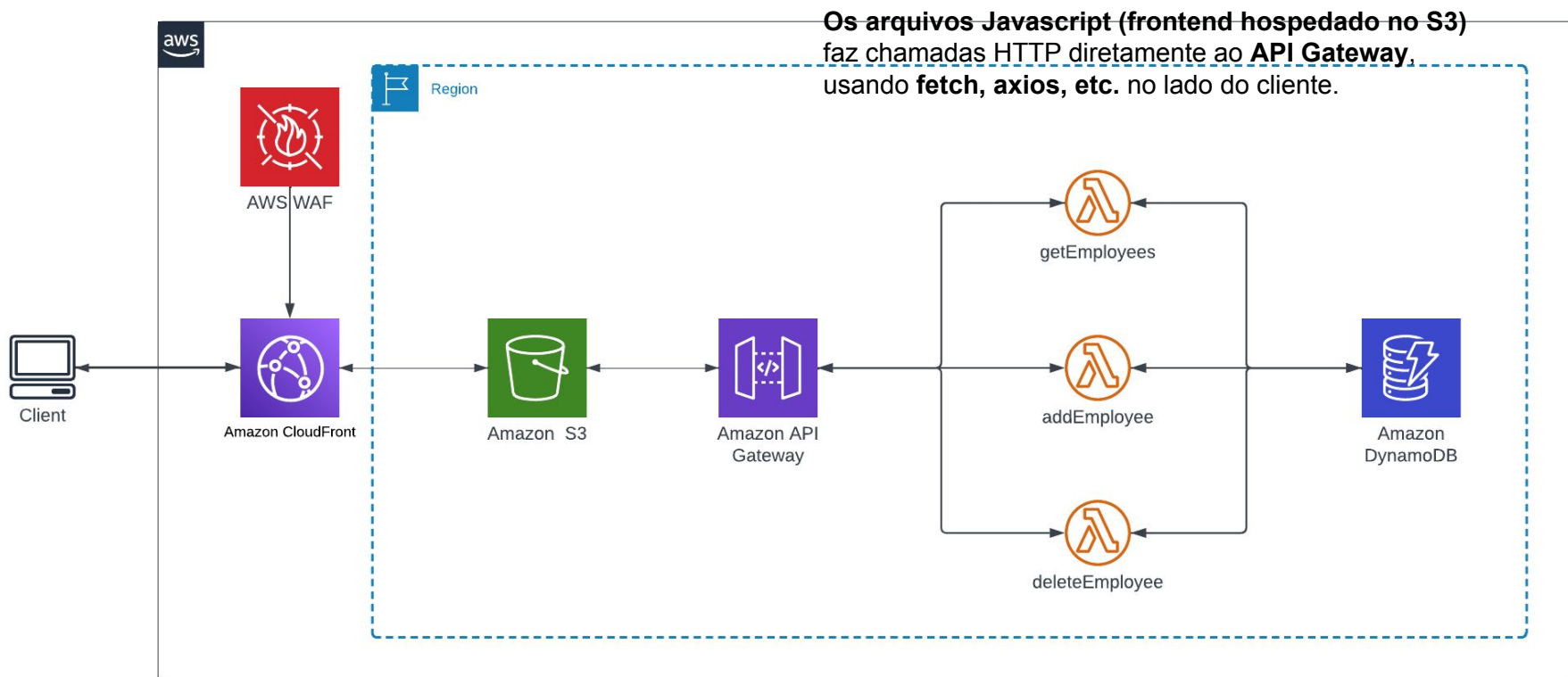


Se o usuário iniciou o pedido de um smartphone há um quinto passo, que é enviar uma notificação Push para o smartphone do usuário confirmando a transação.

Serverless Architecture

- O que mais poderia ser melhorado (ou inventado) com uma arquitetura de software *serverless*?
 - O que acham de um aplicativo para entrega das bebidas por drone?
 - Ou para melhorar os pedidos/distribuição das bebidas para os demais distribuidores (vendas no atacado)?
 - Como poderia ser essa arquitetura?

Serverless Architecture



Web (frontend): arquivos estáticos (HTML/CSS/JS, imagens) colocados em um bucket S3 e servidos globalmente via CloudFront (CDN). O `index.html` é o root do site.

API Gateway + Lambda (Functions): expõe endpoints REST (GET, POST, DELETE etc.). As Lambdas tratam as requisições e implementam a lógica de leitura/escrita.

DynamoDB: banco NoSQL usado para armazenar os dados da aplicação (itens usados pela API).

CloudFront + WAF: CloudFront acelera entrega do frontend; o WAF protege contra ataques/requests maliciosas (o README mostra como configurar regras gerenciadas).

<https://github.com/kemanedonfack/aws-serverless-web-application>

Serverless Architecture

<https://github.com/mikaelvesavuori/get-a-room-ddd-example/blob/main/readme/solution-diagram.png>

Dúvidas?



www.shutterstock.com · 744867163